

UNIVERSIDAD DE CARABOBO
FACULTAD DE CIENCIA Y TECNOLOGÍA (FACYT)
DEPARTAMENTO DE COMPUTACIÓN

Guia de Referencia de MIPS 32

Materia: Arquitectura del Computador (4to Semestre)

Guía Exhaustiva de Código Basada en Patterson & Hennessy (4ta Edición)

1 Estructura Organizativa de Registros en MIPS 32

MIPS utiliza una arquitectura Registro-Registro (RISC). Para codificar correctamente cualquier algoritmo, es mandatorio respetar el contrato de preservación de registros establecido por el hardware.

Nombre	Número	Uso en Arquitectura / Convención de Software	¿Se preserva en llamadas?
\$zero	0	Constante cableada al valor cero (0)	No modificable
\$at	1	Reservado temporalmente para el Ensamblador	No
\$v0-\$v1	2-3	Valores de retorno de funciones evaluadas	No
\$a0-\$a3	4-7	Parámetros de argumentos de entrada	No
\$t0-\$t7	8-15	Registros temporales volátiles	No
\$s0-\$s7	16-23	Registros salvados (Contexto local a proteger)	Sí
\$t8-\$t9	24-25	Registros temporales volátiles adicionales	No
\$k0-\$k1	26-27	Reservados de uso exclusivo del Núcleo del S.O.	No
\$gp	28	Puntero de área global de variables estáticas	Sí
\$sp	29	Puntero de Pila (<i>Stack Pointer</i>)	Sí
\$fp	30	Puntero de Marco de Activación (<i>Frame Pointer</i>)	Sí
\$ra	31	Dirección de Retorno de la función actual (<i>Return Address</i>)	Sí

Cuadro 1: Mapa de Convenciones del Banco de Registros en MIPS.

2 Estructura Esquelética de un Programa en MARS

Todo archivo ejecutable ‘asm’ debe dividir los recursos de memoria lógica mediante directivas claras del compilador.

Listing 1: Plantilla estructural limpia para desarrollo en laboratorios.

```

1  #
   -----
2  # AUTOR: Gabriel Vargas
3  # DESCRIPCIÓN: Estructura base para el desarrollo de proyectos MIPS
4  #
   -----
5

```

```
6 .data
7     # Segmento de Memoria Estática (RAM)
8     # Declaración de variables globales, cadenas ASCII y estructuras
    inicializadas
9     mensaje_inicio: .asciiz "Iniciando ejecución del programa maestro...\n"
10    salto_linea:     .asciiz "\n"
11    alfa_numerico:    .byte 'M'
12    constante_pi:     .word 314159     # Representación entera fija
13
14 .text
15 .globl main
16
17 main:
18     # --- BLOQUE INICIAL DE CONTROL ---
19     la $a0, mensaje_inicio
20     li $v0, 4
21     syscall                # Impresión de bienvenida por consola
22
23     # --- EJECUCIÓN ESPECÍFICA ---
24     # (Pega aquí la lógica de tu algoritmo)
25
26     # --- SALIDA REGLAMENTARIA Y LIMPIA ---
27     li $v0, 10             # Código del sistema para terminar el hilo
    actual
28     syscall
```

3 Capítulo 2.2 a 2.5: Operaciones del Núcleo Aritmético y Lógico

MIPS procesa tres tipos primarios de formatos de instrucción: Tipo R (Registros), Tipo I (Inmediatos) y Tipo J (Saltos). Aquí se muestran todas las variaciones posibles de manipulación aritmética y de bits.

Listing 2: Demostración total de operaciones de tipo aritmético-lógico.

```

1  # === 1. ARITMÉTICA DE ENTEROS ===
2  add  $t0, $s0, $s1      # Suma con desbordamiento controlado: $t0 = $s0 + $s1
3  addu $t1, $s0, $s1      # Suma sin signo (Ignora excepciones de overflow)
4  sub  $t2, $s2, $s3      # Resta básica: $t2 = $s2 - $s3
5  subu $t3, $s2, $s3      # Resta básica sin signo
6
7  # === 2. MANIPULACIÓN CON CONSTANTES (INMEDIATOS) ===
8  addi $t4, $s0, 256      # Suma inmediata con signo: $t4 = $s0 + 256
9  addiu $t5, $s0, -40     # Suma inmediata sin signo (Extiende el signo del
    inmediato)
10
11 # === 3. OPERACIONES DE BITS (MASCARAS) ===
12 and  $t6, $s1, $s2      # AND lógico bit a bit
13 andi $t7, $s1, 0x00FF   # AND inmediato: Extrae o aísla los 8 bits inferiores
    (Máscara de limpieza)
14 or   $s4, $t6, $t7      # OR lógico bit a bit
15 ori  $s5, $t6, 0x0F00   # OR inmediato: Enciende selectivamente grupos de bits
16 xor  $t8, $s1, $s2      # XOR (OR exclusivo): Detecta diferencias de bits
17 nor  $t9, $s0, $zero    # NOR contra el registro cero: Invierte todos los bits
    . Es el operador NOT.
18
19 # === 4. MULTIPLICACIÓN Y DIVISIÓN COMPLETA (Registros Especiales lo y hi) ===
20 mult $s0, $s1           # Multiplica $s0 * $s1. El resultado de 64 bits va a [
    hi, lo]
21 mflo $t0                # Mueve la mitad inferior del producto desde 'lo' a
    $t0
22 mfhi $t1                # Mueve la mitad superior del producto desde 'hi' a
    $t1
23
24 div  $s2, $s3           # Divide $s2 / $s3. Cociente se guarda en 'lo',
    Residuo en 'hi'
25 mflo $s4                # $s4 = Cociente
26 mfhi $s5                # $s5 = Residuo (Operación Módulo)
27
28 # === 5. DESPLAZAMIENTOS M ÁQUINA (SHIFTS) ===
29 sll  $t0, $s1, 3        # Shift Left Logical: Desplaza 3 bits a la izquierda (
    Multiplica por 8)
30 srl  $t1, $s2, 2        # Shift Right Logical: Desplaza 2 bits a la derecha (
    Divide entre 4 sin signo)
31 sra  $t2, $s2, 2        # Shift Right Arithmetic: Desplaza conservando el bit
    de signo superior

```

4 Capítulo 2.3 y 2.8: Modos de Direccionamiento de Memoria

La memoria RAM lee y escribe usando bytes indexados de forma absoluta. Las palabras completas exigen alineación obligatoria de 4 bytes.

Listing 3: Lectura y almacenamiento de estructuras de datos discretas.

```

1  .data
2      vector_enteros:    .word 105, 210, 315, 420
3      vector_caracter:  .byte 'A', 'B', 'C', 'D', 'E'
4      vector_cortos:    .half 12, 24, 36, 48
5
6  .text
7  main_memoria:
8      la $s0, vector_enteros    # Cargar dirección base de la estructura de
9      palabras (32 bits)
10     la $s1, vector_caracter    # Cargar dirección base de la estructura de
11     caracteres (8 bits)
12     la $s2, vector_cortos      # Cargar dirección base de la estructura de
13     cortos (16 bits)
14
15     # LECTURAS (De memoria RAM a Registros CPU)
16     lw $t0, 0($s0)            # Carga vector_enteros[0] -> Desplazamiento base
17     0
18     lw $t1, 4($s0)            # Carga vector_enteros[1] -> Salta 4 bytes
19     adelante
20     lw $t2, 12($s0)           # Carga vector_enteros[3] -> Salta 3 * 4 = 12
21     bytes adelante
22
23     lbu $t3, 1($s1)           # Carga vector_caracter[1] como un byte sin signo
24     (Carácter 'B')
25     lh $t4, 2($s2)            # Carga vector_cortos[1] -> Salta 1 * 2 = 2 bytes
26     adelante
27
28     # ESCRITURAS (De Registros CPU a memoria RAM)
29     addi $t0, $t0, 100        # Modificar el valor local
30     sw $t0, 8($s0)            # Escribe el nuevo valor en vector_enteros[2] (
31     Reemplaza el original)
32
33     li $t5, 'Z'
34     sb $t5, 0($s1)            # Sobrescribe vector_caracter[0] convirtiéndolo
35     en 'Z'

```

5 Capítulo 2.6: Control de Flujo y Mapeo Algorítmico Base 1

5.1 Estructuras Condicionales Compuestas (If-Else / Selectores)

Mapeo de la estructura lógica: `if (A < B) { C = A; } else { C = B; }`

Asignación de registros: `A = $s0, B = $s1, C = $s2.`

Listing 4: Uso clásico de la comparación `slt` para bifurcaciones condicionales.

```

1  slt $t0, $s0, $s1           # Si $s0 < $s1, entonces $t0 = 1; de lo
    contrario $t0 = 0
2  beq $t0, $zero, ej_else     # Si $t0 == 0 (Es decir, A >= B), saltar al
    bloque Else
3
4  # --- BLOQUE THEN ---
5  move $s2, $s0               # C = A
6  j fin_condicional           # Evitar la ejecución del bloque Else
7
8  ej_else:
9  # --- BLOQUE ELSE ---
10 move $s2, $s1               # C = B
11
12 fin_condicional:

```

5.2 Estructuras de Bucles de Iteración (While / For)

Código equivalente en C: `for (int i = 0; i < N; i++) { suma += vector[i]; }`

Asignación: `i = $t0, N = $s0, suma = $s1, dirección de vector = $s2.`

Listing 5: Bucle indexado estándar con cálculo dinámico de offset.

```

1  li $t0, 0                   # Inicializar el contador de ciclos i = 0
2  li $s1, 0                   # Inicializar acumulador suma = 0
3
4  ciclo_for:
5  slt $t1, $t0, $s0           # Verificar si i < N
6  beq $t1, $zero, fin_for     # Si $t1 == 0 (i >= N), romper el bucle e ir al
    final
7
8  # Cálculo del desplazamiento físico dentro del bucle
9  sll $t2, $t0, 2             # $t2 = i * 4 (Cada palabra de entero ocupa 4
    bytes)
10 add $t3, $t2, $s2           # $t3 = Dirección específica física calculada (
    Base + Desplazamiento)
11
12 lw  $t4, 0($t3)              # Cargar dinámicamente el elemento vector[i]
13 add $s1, $s1, $t4           # suma += vector[i]
14
15 addi $t0, $t0, 1             # Incrementar contador: i++
16 j  ciclo_for                 # Evaluar la siguiente iteración
17
18 fin_for:

```

5.3 Conversión Especial: Arreglos Base 1 (Algoritmos) a Base 0 (Hardware)

En las evaluaciones teóricas de algoritmos en FACYT se suele asumir que los índices de los arreglos inician en **1**. En MIPS, las posiciones físicas de memoria siempre inician estrictamente en **0**. El modelo matemático exacto para resolver este desajuste es:

$$\text{Dirección Física} = \text{Dirección Base} + [(\text{Índice Lógico}_{\text{Base } 1} - 1) \times \text{Tamaño de la Palabra}] \quad (1)$$

Listing 6: Rutina de adaptación segura para índices de pseudocódigo en Base 1.

```

1  # Parámetros: $s0 = Dirección Base del Arreglo en RAM, $s1 = Índice "i" en Base
   # 1 deseado.
2  # Requerimiento: Leer el dato simulado arreglo[i]
3
4  addi $t0, $s1, -1          # Ajuste matemático inmediato: Convertir a Base 0 (
   $t0 = i - 1)
5  bltz $t0, error_limite    # Protección: Si el índice ajustado es menor a 0,
   abortar por violación
6
7  sll  $t1, $t0, 2          # Multiplicar el índice ajustado por 4 bytes (Tamaño
   de palabra)
8  add  $t2, $t1, $s0        # Calcular la dirección real combinada
9  lw   $s2, 0($t2)         # Cargar con éxito el dato equivalente al algoritmo
   teórico
10
11 j  fin_acceso
12
13 error_limite:
14     # Manejo preventivo de excepciones de desbordamiento de arreglos
15 fin_acceso:

```

6 Capítulo 2.7: Administración de Procedimientos y Gestión de la Pila

6.1 Estructura de Funciones Anidadas No-Hoja

Una función No-Hoja ejecuta sub-llamadas internas mediante la instrucción ‘jal’. Esto causa la pérdida destructiva inmediata del contenido del registro ‘ra’. Por lo tanto, la función debe crear su propio marco de pila seguro en memoria.

Listing 7: Plantilla estándar obligatoria para el prólogo y epílogo del manejo de la Pila.

```

1  #
   -----
2  # FUNCIÓN EJEMPLO: Procesa datos y realiza una llamada anidada interna
3  #
   -----
4  funcion_compleja:
5      # === 1. PRÓLOGO DE LA FUNCIÓN ===
6      addi $sp, $sp, -8          # Reservamos espacio para 2 palabras continuas
   (8 bytes)
7      sw    $ra, 4($sp)         # Guardamos la dirección de retorno del llamador
   original
8      sw    $s0, 0($sp)         # Salvamos el valor previo del registro $s0
9
10     # === 2. CUERPO OPERATIVO ===
11     move   $s0, $a0            # Protegemos el primer argumento de entrada en
   $s0
12     add    $a0, $s0, $a1       # Preparamos un parámetro modificado para la sub
   -función
13
14     jal    sub_rutina_interna   # Ejecutar la llamada anidada. Al hacer esto se
   altera $ra de forma local.
15                                     # El resultado parcial de la sub-rutina se
   recibe en el registro $v0
16
17     mul    $v0, $v0, $s0       # Multiplicar el resultado parcial por el valor
   inicial guardado en $s0
18
19     # === 3. EPÍLOGO DE LA FUNCIÓN ===
20     lw     $s0, 0($sp)         # Restauramos el valor original e intacto de $s0
   del llamador
21     lw     $ra, 4($sp)         # Recuperamos la dirección de retorno REAL hacia
   main
22     addi   $sp, $sp, 8         # Liberamos los 8 bytes sumando de vuelta al
   puntero de pila
23
24     jr     $ra                # Retornar de forma exitosa y segura

```

6.2 Implementación de un Algoritmo Recursivo Puro (Factorial)

Demostración avanzada de recursión matemática controlada por pila.

Listing 8: Código máquina para el cálculo recursivo de N!.

```

1  # Argumento de Entrada: $a0 = Número entero N
2  # Retorno: $v0 = Resultado del factorial (N!)

```

```

3 factorial:
4     addi $sp, $sp, -8           # Reservar espacio en la pila para el sub-árbol
5     sw   $ra, 4($sp)           # Salvar dirección de retorno
6     sw   $a0, 0($sp)           # Salvar el valor del argumento actual N
7
8     slti $t0, $a0, 1           # Evaluar caso base: Es N < 1?
9     beq  $t0, $zero, ir_recurso # Si N >= 1, continuar con las llamadas
    descendientes
10
11     # --- CASO BASE ALCANZADO (0! = 1) ---
12     li   $v0, 1                # Cargar el resultado neutro del inicio
13     addi $sp, $sp, 8           # Liberar pila del nivel actual sin leer memoria
14     jr   $ra                  # Iniciar el retorno hacia arriba
15
16 ir_recurso:
17     addi $a0, $a0, -1          # Decrementar el argumento de control: (N - 1)
18     jal  factorial             # Llamada recursiva interna: factorial(N-1)
19     # Al regresar, $v0 contiene el resultado
20     acumulado acumulado
21     lw   $a0, 0($sp)           # Restaurar el valor original de N asignado a
    este nivel
22     lw   $ra, 4($sp)           # Restaurar la dirección de retorno de este
    nivel
23     addi $sp, $sp, 8           # Liberar el espacio del marco de pila actual
24
25     mul  $v0, $a0, $v0         # Multiplicación combinada: N * factorial(N - 1)
26     jr   $ra                  # Volver al llamador del nivel superior

```


7 Algorítmica Avanzada para Evaluaciones y Proyectos

7.1 Ordenamiento de Arreglos (Procedimiento Completo Sort y Swap)

El algoritmo de ordenamiento por intercambio (*Bubble / Exchange Sort*) representa el ejemplo cumbre de un procedimiento No-Hoja con dos bucles condicionales anidados según el texto guía de Patterson & Hennessy.

Listing 9: Código máquina MIPS completo para la rutina sort y sub-rutina swap.

```

1  #
   -----
2  # PROCEDIMIENTO PRINCIPAL DE ORDENAMIENTO: sort (Función No-Hoja)
3  # Argumentos: $a0 = Dirección Base del Arreglo, $a1 = Tamaño N
4  # Mapeo: $s0 = i, $s1 = j, $s2 = v (dirección base), $s3 = n (tamaño)
5  #
   -----
6  sort:
7      # --- PRÓLOGO: Reservar espacio en la Pila (Stack Frame) ---
8      addi $sp, $sp, -20          # Espacio para resguardar 5 palabras (20 bytes)
9      sw   $ra, 16($sp)          # VITAL ! Guardar retorno a main
10     sw   $s3, 12($sp)          # Guardar contexto de $s3
11     sw   $s2, 8($sp)           # Guardar contexto de $s2
12     sw   $s1, 4($sp)           # Guardar contexto de $s1
13     sw   $s0, 0($sp)           # Guardar contexto de $s0
14
15     # --- INICIALIZACIÓN ---
16     move $s2, $a0               # Copiar dirección base de v para protegerla
17     move $s3, $a1               # Copiar el tamaño N para protegerlo
18     move $s0, $zero             # i = 0 (Inicializa el lazo externo)
19
20     # === PRIMER LAZO FOR (Externo) ===
21     for_externo_test:
22         slt $t0, $s0, $s3        # i < n?
23         beq $t0, $zero, exit_externo # Si $t0 == 0 (i >= n), ir al epílogo final
24
25         addi $s1, $s0, -1        # Inicialización del lazo interno: j = i - 1
26
27     # === SEGUNDO LAZO FOR (Interno) ===
28     for_interno_test:
29         # Condición Parte 1: j >= 0?
30         slti $t0, $s1, 0         # Si j < 0 entonces $t0 = 1
31         bne $t0, $zero, exit_interno # LÓGICA INVERTIDA: Si j < 0, sale del lazo interno
32
33         # Condición Parte 2: v[j] > v[j + 1]?
34         sll $t1, $s1, 2          # $t1 = j * 4 (Convertir índice a bytes)
35         add $t2, $s2, $t1        # $t2 = v + (j * 4) -> Dirección física de v[j]
36
37         lw  $t3, 0($t2)          # $t3 = v[j]
38         lw  $t4, 4($t2)          # $t4 = v[j + 1] (4 bytes más adelante)
39
40         slt $t0, $t4, $t3        # v[j + 1] < v[j]? (Equivalente a v[j] > v[j+1])

```

```

41     beq    $t0, $zero, exit_interno # LÓGICA INVERTIDA: Si no es mayor, rompe el
      ciclo
42
43     # --- CUERPO DEL FOR INTERNO (Llamada a Swap) ---
44     move   $a0, $s2                # Parámetro 1: Dirección base v
45     move   $a1, $s1                # Parámetro 2: Índice j actual
46     jal    swap                    # Saltar a sub-rutina swap. Al regresar ejecuta
      la sig. línea
47
48     # --- ACTUALIZACIÓN DEL FOR INTERNO ---
49     addi   $s1, $s1, -1            # j--
50     j      for_interno_test        # Volver a evaluar el lazo interno
51
52 exit_interno:
53     # --- ACTUALIZACIÓN DEL FOR EXTERNO ---
54     addi   $s0, $s0, 1            # i++
55     j      for_externo_test        # Volver a evaluar el lazo externo
56
57 exit_externo:
58     # --- EPÍLOGO: Restaurar la Pila y Retornar ---
59     lw     $s0, 0($sp)            # Restaurar $s0 original del llamador
60     lw     $s1, 4($sp)            # Restaurar $s1
61     lw     $s2, 8($sp)            # Restaurar $s2
62     lw     $s3, 12($sp)           # Restaurar $s3
63     lw     $ra, 16($sp)           # Recuperar el verdadero $ra que apunta al main
64     addi   $sp, $sp, 20           # Liberar de forma exacta los 20 bytes
65     jr     $ra                    # Retornar de forma segura
66
67
68 #
      -----
69 # PROCEDIMIENTO AUXILIAR: swap (Función Hoja)
70 # Argumentos: $a0 = Dirección Base v, $a1 = Índice j
71 #
      -----
72 swap:
73     sll    $t0, $a1, 2            # $t0 = j * 4
74     add    $t1, $a0, $t0          # $t1 = Dirección física de v[j]
75
76     lw     $t2, 0($t1)            # $t2 = v[j]
77     lw     $t3, 4($t1)            # $t3 = v[j + 1]
78
79     sw     $t3, 0($t1)            # v[j] = v[j + 1] (Escritura cruzada)
80     sw     $t2, 4($t1)            # v[j + 1] = v[j]
81
82     jr     $ra                    # Retorna directo a la línea interna de sort

```

7.2 Acceso y Mapeo de Matrices Bidimensionales (Filas y Columnas)

Para procesar una matriz estática guardada en formato lineal por filas (*Row-Major Order*), la fórmula matemática para calcular el desplazamiento del elemento `Matriz[fila][columna]` es:

$$\text{Offset en Bytes} = (\text{Fila} \times \text{Numero de Columnas Totales} + \text{Columna}) \times \text{Tamaño del Dato} \quad (2)$$

Listing 10: Rutina de extracción de elementos en matrices dinámicas de enteros.

```

1  .data
2      # Matriz simulada de 3x4 (3 filas, 4 columnas) con enteros secuenciales
3      matriz_datos: .word 11, 12, 13, 14      # Fila 0
4                          .word 21, 22, 23, 24      # Fila 1
5                          .word 31, 32, 33, 34      # Fila 2
6      columnas_tot: .word 4                    # Constante de ancho estructural
7
8  .text
9  main_matrices:
10     la $s0, matriz_datos      # Dirección base de la estructura en RAM
11     lw $s1, columnas_tot      # $s1 = 4 (Ancho total de la matriz)
12
13     # Coordinadas lógicas del elemento que deseamos extraer (Ej: Fila 1,
14     # Columna 3 -> Valor 24)
15     li $s2, 1                 # Fila objetivo = 1
16     li $s3, 3                 # Columna objetivo = 3 (Buscamos matriz_datos
17     # [1][3] -> 24)
18
19     # Implementación de la fórmula física de direccionamiento
20     mul $t0, $s2, $s1          # $t0 = Fila objetivo * Columnas totales (1 * 4
21     # = 4)
22     add $t0, $t0, $s3          # $t0 = Linealización de coordenadas ($t0 = 4 +
23     # 3 = 7)
24
25     sll $t1, $t0, 2            # Convertir a bytes multiplicando por 4 ($t1 = 7
26     # * 4 = 28 bytes de offset)
27     add $t2, $t1, $s0          # $t2 = Dirección física exacta en la RAM
28
29     lw $s4, 0($t2)             # $s4 = Contenido extraído de la celda deseada (
30     # Valor = 24)

```

8 Catálogo Completo de Servicios de Interrupción (Syscalls)

Carga el identificador en \$v0 y pasa los parámetros requeridos antes de activar el servicio.

Servicio	Código (\$v0)	Argumentos de Entrada requeridos	Resultado del Sistema
Imprimir Entero	1	\$a0 = Contiene el valor numérico entero	Muestra el número en consola
Imprimir Cadena	4	\$a0 = Dirección base de la cadena (.asciiz)	Muestra el texto en consola
Leer Entero	5	Ninguno	Captura entrada; resultado en \$v0
Leer Cadena	8	\$a0 = Dirección del Buffer, \$a1 = Tamaño límite	Almacena el texto en memoria
Asignar Memoria	9	\$a0 = Cantidad de bytes requeridos	Devuelve dirección asignada en \$v0
Terminar Programa	10	Ninguno	Cierra el hilo del programa de for

Cuadro 2: Servicios de llamadas de consola indispensables para laboratorios en MARS.